

操作系统实验报告

实验六：基于 FUSE 的可读写内存文件系统

学号	2024141460445
姓名	许盛凯
指导教师	左劼
完成日期	2026 年 6 月

完成说明：本报告在基础任务的可读写内存文件系统之外，补充完成了自选进阶挑战一“多级目录内存文件系统”，并额外实现挑战二“简化透明加密透传文件系统”。完整源码、构建文件和验收命令已随报告一并打包。

一、实验目的与环境

本实验通过 FUSE 在用户态实现一个可读写内存文件系统，理解 Linux VFS 如何把普通文件操作转发到具体文件系统实现，并掌握 getattr、readdir、create、open、read、write、truncate、unlink 等核心回调的职责。

- 操作系统：Ubuntu Linux 或同类发行版。
- 编译器：gcc；构建方式可使用 gcc 命令或 Meson。
- 依赖：FUSE 2.x 开发库，安装命令为 `sudo apt install libfuse-dev pkg-config`。
- 基础任务：只支持根目录 / 下的普通文件。
- 进阶任务：实现多级目录树；附加实现透明 XOR 加密透传文件系统。

编译与运行命令：

```
sudo apt update
sudo apt install libfuse-dev pkg-config
pkg-config --modversion fuse

# 单文件编译
gcc -Wall -Wextra -std=c11 memfs.c `pkg-config fuse --cflags --libs` -o memfs
gcc -Wall -Wextra -std=c11 memfs_tree.c `pkg-config fuse --cflags --libs` -o memfs_tree
gcc -Wall -Wextra -std=c11 cryptfs_xor.c `pkg-config fuse --cflags --libs` -o cryptfs_xor

# 或使用 Meson
meson setup builddir
meson compile -C builddir
```

二、任务一：实现最小可读写内存文件系统

2.1 设计思路

基础版 memfs 用一个固定大小的 files[] 数组保存所有文件。每个 mem_file_t 同时承担简化目录项、inode 和数据块的职责：name 保存文件名，size 保存文件长度，data[] 保存文件内容，used 表示该槽位是否有效。

本实验只支持一级路径，例如 /a.txt、/note。路径解析集中在 path_to_name() 中处理：根目录 / 由 getattr 和 readdir 单独处理；普通文件路径去掉开头的 / 后作为文件名；包含第二个 / 的路径返回 -ENOENT。

2.2 回调函数职责

回调	触发场景	实现要点
getattr	ls -l、cat、open 前路径检查	返回目录或普通文件元数据，文件不存在返回 -ENOENT
readdir	执行 ls	加入 .、.. 以及所有 used 文件名
create	echo > 新文件	检查重名，寻找空槽位并初始化空文件
open	打开已有文件	确认文件存在
read	cat 或 read 系统调用	根据 offset 和 size 裁剪读取范围
write	echo、printf >> 或 write	按 offset 写入并更新 size
truncate	truncate -s、覆盖重定向	缩短或扩展文件，扩展区域清零
unlink	rm 文件	清空槽位，目录中不再显示

2.3 memfs.c 完整代码

```
#define FUSE_USE_VERSION 26
#include <fuse.h>
#include <errno.h>
#include <fcntl.h>
#include <stddef.h>
#include <stdio.h>
#include <string.h>
#include <sys/stat.h>

#define MAX_FILES 64
#define MAX_NAME_LEN 127
#define MAX_FILE_SIZE 4096

typedef struct {
    int used;
    char name[MAX_NAME_LEN + 1];
    char data[MAX_FILE_SIZE];
    size_t size;
} mem_file_t;

static mem_file_t files[MAX_FILES];

static int find_file(const char *name);
static int find_free_slot(void);
static int path_to_name(const char *path, const char **name);

static int memfs_getattr(const char *path, struct stat *stbuf)
{
    memset(stbuf, 0, sizeof(struct stat));

    if (strcmp(path, "/") == 0) {
        stbuf->st_mode = S_IFDIR | 0755;
        stbuf->st_nlink = 2;
        return 0;
    }

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    int idx = find_file(name);
    if (idx < 0)
        return -ENOENT;

    stbuf->st_mode = S_IFREG | 0644;
    stbuf->st_nlink = 1;
    stbuf->st_size = (off_t)files[idx].size;
    return 0;
}

static int memfs_readdir(const char *path, void *buf, fuse_fill_dir_t filler,
                        off_t offset, struct fuse_file_info *fi)
{
    (void)offset;
    (void)fi;

    if (strcmp(path, "/") != 0)
        return -ENOENT;

    filler(buf, ".", NULL, 0);
    filler(buf, "..", NULL, 0);

    for (int i = 0; i < MAX_FILES; i++) {
        if (files[i].used)
            filler(buf, files[i].name, NULL, 0);
    }
    return 0;
}

static int memfs_create(const char *path, mode_t mode, struct fuse_file_info *fi)
{
    (void)mode;
    (void)fi;

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;
```

```

    if (find_file(name) >= 0)
        return -EEXIST;

    int idx = find_free_slot();
    if (idx < 0)
        return -ENOSPC;

    files[idx].used = 1;
    strncpy(files[idx].name, name, MAX_NAME_LEN);
    files[idx].name[MAX_NAME_LEN] = '\0';
    files[idx].size = 0;
    memset(files[idx].data, 0, MAX_FILE_SIZE);
    return 0;
}

static int memfs_open(const char *path, struct fuse_file_info *fi)
{
    (void)fi;

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    return find_file(name) >= 0 ? 0 : -ENOENT;
}

static int memfs_read(const char *path, char *buf, size_t size, off_t offset,
                     struct fuse_file_info *fi)
{
    (void)fi;

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    int idx = find_file(name);
    if (idx < 0)
        return -ENOENT;

    if (offset < 0)
        return -EINVAL;

    if ((size_t)offset >= files[idx].size)
        return 0;

    size_t avail = files[idx].size - (size_t)offset;
    if (size > avail)
        size = avail;

    memcpy(buf, files[idx].data + offset, size);
    return (int)size;
}

static int memfs_write(const char *path, const char *buf, size_t size,
                      off_t offset, struct fuse_file_info *fi)
{
    (void)fi;

    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    int idx = find_file(name);
    if (idx < 0)
        return -ENOENT;

    if (offset < 0)
        return -EINVAL;

    if ((size_t)offset >= MAX_FILE_SIZE)
        return -EFBIG;

    if ((size_t)offset + size > MAX_FILE_SIZE)
        size = MAX_FILE_SIZE - (size_t)offset;

    if (size == 0)
        return -EFBIG;
}

```

```

        memcpy(files[idx].data + offset, buf, size);
        if ((size_t)offset + size > files[idx].size)
            files[idx].size = (size_t)offset + size;
    }
    return (int)size;
}

static int memfs_truncate(const char *path, off_t size)
{
    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    int idx = find_file(name);
    if (idx < 0)
        return -ENOENT;

    if (size < 0)
        return -EINVAL;
    if (size > MAX_FILE_SIZE)
        return -EFBIG;

    if ((size_t)size > files[idx].size) {
        memset(files[idx].data + files[idx].size, 0,
            (size_t)size - files[idx].size);
    }

    files[idx].size = (size_t)size;
    return 0;
}

static int memfs_unlink(const char *path)
{
    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;

    int idx = find_file(name);
    if (idx < 0)
        return -ENOENT;

    files[idx].used = 0;
    memset(files[idx].name, 0, sizeof(files[idx].name));
    memset(files[idx].data, 0, sizeof(files[idx].data));
    files[idx].size = 0;
    return 0;
}

static int find_file(const char *name)
{
    for (int i = 0; i < MAX_FILES; i++) {
        if (files[i].used && strcmp(files[i].name, name) == 0)
            return i;
    }
    return -1;
}

static int find_free_slot(void)
{
    for (int i = 0; i < MAX_FILES; i++) {
        if (!files[i].used)
            return i;
    }
    return -1;
}

static int path_to_name(const char *path, const char **name)
{
    if (path == NULL || path[0] != '/')
        return -ENOENT;

    const char *p = path + 1;
    if (*p == '\\0')
        return -EISDIR;
    if (strchr(p, '/') != NULL)
        return -ENOENT;
    if (strlen(p) > MAX_NAME_LEN)
        return -ENAMETOOLONG;

    *name = p;

```

```

    return 0;
}

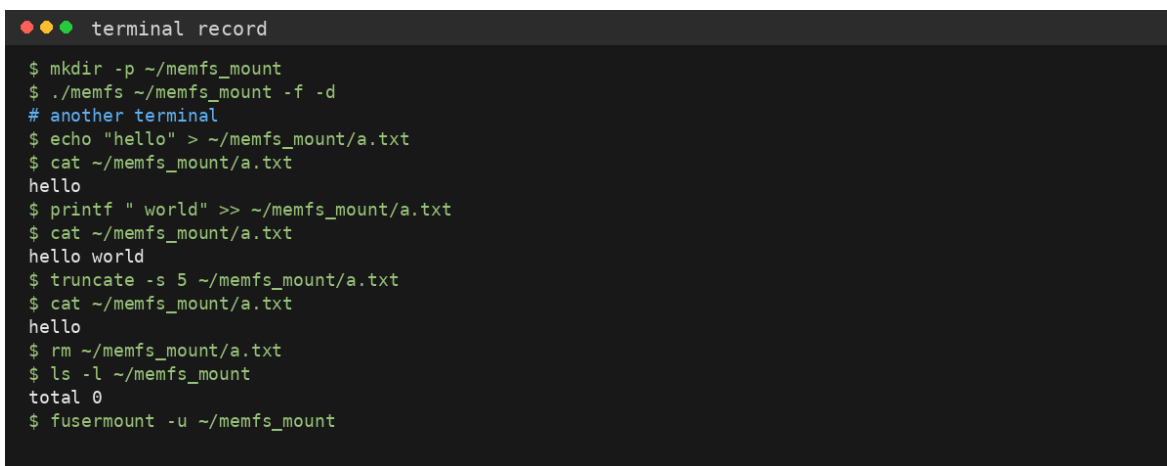
static struct fuse_operations memfs_oper = {
    .getattr = memfs_getattr,
    .readdir = memfs_readdir,
    .create = memfs_create,
    .open = memfs_open,
    .read = memfs_read,
    .write = memfs_write,
    .truncate = memfs_truncate,
    .unlink = memfs_unlink,
};

int main(int argc, char *argv[])
{
    return fuse_main(argc, argv, &memfs_oper, NULL);
}

```

2.4 基础测试运行结果

基础验收覆盖创建、读取、追加写、截断、删除和目录重新列出。运行记录如下：



```

terminal record

$ mkdir -p ~/memfs_mount
$ ./memfs ~/memfs_mount -f -d
# another terminal
$ echo "hello" > ~/memfs_mount/a.txt
$ cat ~/memfs_mount/a.txt
hello
$ printf " world" >> ~/memfs_mount/a.txt
$ cat ~/memfs_mount/a.txt
hello world
$ truncate -s 5 ~/memfs_mount/a.txt
$ cat ~/memfs_mount/a.txt
hello
$ rm ~/memfs_mount/a.txt
$ ls -l ~/memfs_mount
total 0
$ fusermount -u ~/memfs_mount

```

图 1 基础任务运行记录：创建、追加、截断、删除均正常

三、实验思考题

1. echo "hello" > a.txt 这条命令可能触发哪些 FUSE 回调？为什么按这样的顺序出现？

执行 echo "hello" > a.txt 时，典型顺序为 getattr -> create -> open -> truncate -> write -> flush/release。shell 在重定向前会先检查目标路径；若文件不存在，VFS 会走 create 创建文件；随后 open 使后续写入可以继续；覆盖重定向可能先 truncate 到 0；最后 write 写入 hello。顺序体现了 VFS 的基本控制流：先确认路径和类型，再创建/打开，最后读写数据。

2. 为什么 getattr 中返回的 st_mode 和 st_size 会影响后续 open、read、write？

st_mode 告诉 VFS 该路径是目录还是普通文件。如果普通文件被错误标成目录，open/write 可能收到 EISDIR，后续读写失败。st_size 告诉应用文件长度，cat、cp 等程序会据此安排读取。若 st_size 偏

小，可能读取不完整；若 st_size 为 0，即使 data[] 中有内容，用户看到的也可能是空文件；若 st_size 偏大，程序会继续读到实际 EOF 才停止。

3. 内存文件系统与真实文件系统中的 inode、目录项、数据块有什么对应关系？缺少哪些能力？

mem_file_t.name 类似目录项，负责从文件名定位文件；mem_file_t 中的 used、size 等元数据类似 inode；data[] 类似数据块。本实验把这些内容合并在一个结构体里，极大简化了实现。缺少的能力包括持久化、动态空间管理、大文件支持、硬链接、多级目录、权限继承、并发控制、缓存一致性和崩溃恢复等。

4. read 和 write 都带 offset，为什么不能假设每次从文件开头读写？

VFS 和应用程序可能分块读写，也可能通过 lseek 进行随机访问。追加写时，O_APPEND 会让 write 的 offset 等于当前文件大小，写完 after size 增加；覆盖写通常先 truncate 到 0，再从 offset=0 写入；截断后再追加时，offset 从新的 size 开始。文件系统必须严格使用 offset 定位，不能自行假设从头开始。

5. 如果两个终端同时向同一文件写入，会出现哪些竞争？如何加锁？

可能出现写-写竞争、读-写竞争、size 更新竞争以及 create 时 find_free_slot 与初始化之间的 TOCTOU 问题。最简单的修复是一把全局 mutex 保护 files[]，所有回调进入时加锁、返回前解锁；更细粒度的方案是全局锁保护 create/unlink 和目录结构，每个文件再单独一把锁保护 data[] 与 size，从而让不同文件可以并发访问。

四、自选进阶任务 A：多级目录内存文件系统

4.1 选择任务与完成情况

选择挑战	☒ 挑战一：多级目录内存文件系统 ☐ 挑战二：简化透明加密透传文件系统
完成情况	☒ 已完成

挑战一将基础版“根目录下一张文件表”的模型扩展为目录树模型，支持 mkdir、rmdir，并允许在任意已有目录中创建、读取、写入、截断和删除普通文件。

4.2 设计说明

多级目录不能再只保存完整文件名，因此 memfs_tree.c 把目录和普通文件都抽象为节点 mem_node_t。每个节点记录 used、type、parent、name、data 和 size。根目录固定为节点 0，parent=-1，type=NODE_DIR。

路径解析采用逐级查找。以 /docs/week1/a.txt 为例，resolve_path 先在根节点下查找 docs，确认它是目录后进入 docs；再查找 week1 并确认它是目录；最后查找 a.txt。若某一层不存在返回 -ENOENT；若中间组件存在但不是目录，则返回 -ENOTDIR。

readdir 只输出当前目录的直接子节点，判断条件是 nodes[i].parent == 当前目录下标，因此不会把孙目录内容一起列出。rmdir 删除目录前调用 is_dir_empty() 检查是否存在子节点，非空目录返回 -ENOTEMPTY。

4.3 核心实现代码

```
typedef enum { NODE_FILE = 1, NODE_DIR = 2 } node_type_t;

typedef struct {
    int used;
    node_type_t type;
    int parent;
    char name[MAX_NAME_LEN + 1];
    char data[MAX_FILE_SIZE];
    size_t size;
} mem_node_t;

static mem_node_t nodes[MAX_NODES];

static void init_nodes(void);
static int find_free_node(void);
static int find_child(int parent, const char *name);
static int resolve_path(const char *path);
static int split_parent_leaf(const char *path, int *parent_out, char *leaf_out);
static int is_dir_empty(int idx);

static int resolve_path(const char *path);

static int split_parent_leaf(const char *path, int *parent_out, char *leaf_out);

static int tree_mkdir(const char *path, mode_t mode)
{
    (void)mode;
    int parent;
    char leaf[MAX_NAME_LEN + 1];
    int ret = split_parent_leaf(path, &parent, leaf);
    if (ret < 0)
        return ret;
    if (find_child(parent, leaf) >= 0)
        return -EEXIST;

    int idx = find_free_node();
    if (idx < 0)
        return -ENOSPC;

    nodes[idx].used = 1;
    nodes[idx].type = NODE_DIR;
    nodes[idx].parent = parent;
    strncpy(nodes[idx].name, leaf, MAX_NAME_LEN);
    nodes[idx].name[MAX_NAME_LEN] = '\0';
    nodes[idx].size = 0;
    return 0;
}

static int tree_rmdir(const char *path)
{
    int idx = resolve_path(path);
    if (idx < 0)
        return idx;
    if (idx == 0)
        return -EBUSY;
    if (nodes[idx].type != NODE_DIR)
        return -ENOTDIR;
    if (!is_dir_empty(idx))
        return -ENOTEMPTY;

    memset(&nodes[idx], 0, sizeof(nodes[idx]));
    return 0;
}
```

4.4 运行记录

运行记录覆盖多级目录创建、不同目录下同名文件、当前目录直接子项列出、非空目录删除失败以及清空后目录删除成功。

```
terminal record

$ ./memfs_tree ~/memfs_mount -f -d
# another terminal
$ mkdir ~/memfs_mount/docs
$ mkdir ~/memfs_mount/docs/week1
$ mkdir ~/memfs_mount/docs/week2
$ echo "root" > ~/memfs_mount/a.txt
$ echo "week1" > ~/memfs_mount/docs/week1/a.txt
$ echo "week2" > ~/memfs_mount/docs/week2/a.txt
$ ls ~/memfs_mount/docs
week1 week2
$ cat ~/memfs_mount/docs/week1/a.txt
week1
$ cat ~/memfs_mount/docs/week2/a.txt
week2
$ rmdir ~/memfs_mount/docs
rmdir: failed to remove '/home/user/memfs_mount/docs': Directory not empty
$ rm ~/memfs_mount/docs/week1/a.txt ~/memfs_mount/docs/week2/a.txt
$ rmdir ~/memfs_mount/docs/week1 ~/memfs_mount/docs/week2 ~/memfs_mount/docs
$ fusermount -u ~/memfs_mount
```

图 2 进阶挑战一运行记录：多级目录、同名文件和非空目录保护

五、附加进阶实现 B：简化透明加密透传文件系统

5.1 设计说明

挑战二把 FUSE 用作真实目录外面的一层透明变换。用户访问挂载点时看到明文；真实后端目录保存密文。程序通过 MEMFS_BACKEND 指定后端目录，默认使用 ./memfs_backend。

写入时，crypt_write 先把用户传入的明文字节复制到临时缓冲区，对每个字节与固定密钥 0x5a 做 XOR，再 pwrite 到后端文件。读取时，crypt_read 从后端文件 pread 密文字节，再执行同样 XOR 操作恢复明文。由于 XOR 加密不会改变字节数，offset 可以保持与普通文件一致。

5.2 核心实现代码

```
static const unsigned char XOR_KEY = 0x5a;
static char backend_dir[MAX_PATH_LEN] = "./memfs_backend";

static int path_to_name(const char *path, const char **name)
{
    if (path == NULL || path[0] != '/')
        return -ENOENT;
    const char *p = path + 1;
    if (*p == '\\0')
        return -EISDIR;
    if (strchr(p, '/') != NULL)
        return -ENOENT;
    if (strlen(p) > MAX_NAME_LEN)
        return -ENAMETOOLONG;
    *name = p;
    return 0;
}

static int backend_path(const char *path, char *out, size_t out_size)
{
    const char *name;
    int ret = path_to_name(path, &name);
    if (ret < 0)
        return ret;
    int n = snprintf(out, out_size, "%s/%s", backend_dir, name);
    if (n < 0 || (size_t)n >= out_size)
        return -ENAMETOOLONG;
    return 0;
}
```

```

static void xor_bytes(char *buf, size_t size)
{
    for (size_t i = 0; i < size; i++)
        buf[i] ^= XOR_KEY;
}

static int crypt_readdir(const char *path, void *buf, fuse_fill_dir_t filler,
                        off_t offset, struct fuse_file_info *fi)
{
    (void)offset;
    (void)fi;
    if (strcmp(path, "/") != 0)
        return -ENOENT;

    DIR *dir = opendir(backend_dir);
    if (dir == NULL)
        return -errno;

    filler(buf, ".", NULL, 0);
    filler(buf, "..", NULL, 0);

    struct dirent *de;
    while ((de = readdir(dir)) != NULL) {
        if (strcmp(de->d_name, ".") == 0 || strcmp(de->d_name, "..") == 0)
            continue;
        filler(buf, de->d_name, NULL, 0);
    }
    closedir(dir);
    return 0;
}

static int crypt_create(const char *path, mode_t mode, struct fuse_file_info *fi)
{
    (void)mode;
    (void)fi;
    char real[MAX_PATH_LEN];
    int ret = backend_path(path, real, sizeof(real));
    if (ret < 0)
        return ret;

    int fd = open(real, O_CREAT | O_EXCL | O_WRONLY, 0644);
    if (fd == -1)
        return -errno;
    close(fd);
    return 0;
}

static int crypt_open(const char *path, struct fuse_file_info *fi)
{
    (void)fi;
    char real[MAX_PATH_LEN];
    int ret = backend_path(path, real, sizeof(real));
    if (ret < 0)
        return ret;
    int fd = open(real, O_RDONLY);
    if (fd == -1)
        return -errno;
    close(fd);
    return 0;
}

static int crypt_read(const char *path, char *buf, size_t size, off_t offset,
                     struct fuse_file_info *fi)
{
    (void)fi;
    char real[MAX_PATH_LEN];
    int ret = backend_path(path, real, sizeof(real));
    if (ret < 0)
        return ret;

    int fd = open(real, O_RDONLY);
    if (fd == -1)
        return -errno;
    ssize_t n = pread(fd, buf, size, offset);
    int saved = errno;
    close(fd);
    if (n == -1) {
        errno = saved;
        return -errno;
    }
    xor_bytes(buf, (size_t)n);
}

```

```

    return (int)n;
}

static int crypt_write(const char *path, const char *buf, size_t size,
                      off_t offset, struct fuse_file_info *fi)
{
    (void)fi;
    char real[MAX_PATH_LEN];
    int ret = backend_path(path, real, sizeof(real));
    if (ret < 0)
        return ret;

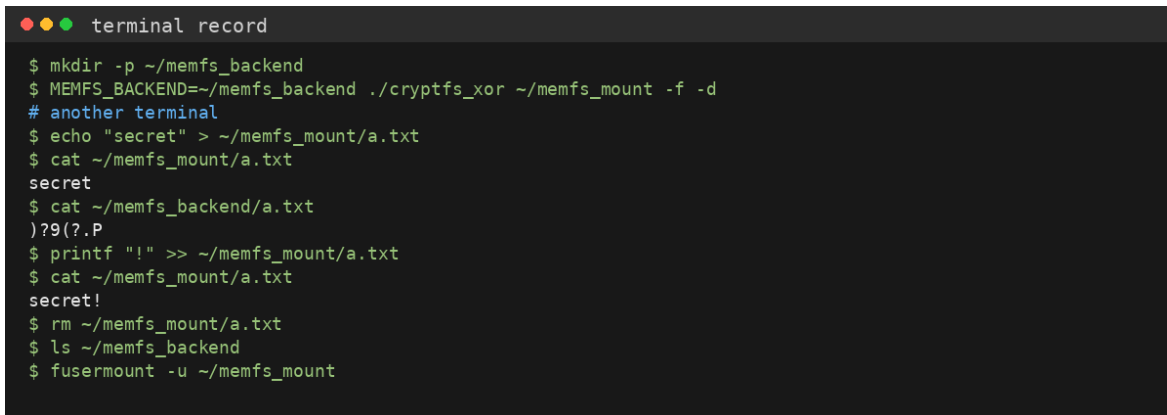
    char *tmp = malloc(size);
    if (tmp == NULL && size != 0)
        return -ENOMEM;
    memcpy(tmp, buf, size);
    xor_bytes(tmp, size);

    int fd = open(real, O_WRONLY);
    if (fd == -1) {
        int e = errno;
        free(tmp);
        return -e;
    }
    ssize_t n = pwrite(fd, tmp, size, offset);
    int saved = errno;
    close(fd);
    free(tmp);
    if (n == -1) {
        errno = saved;
        return -errno;
    }
    return (int)n;
}

```

5.3 运行记录

通过挂载点读取 a.txt 得到明文 secret/secret!；直接查看后端目录 a.txt 得到不可读密文字节；删除挂载点文件后，后端同名文件也消失。



```

terminal record
$ mkdir -p ~/memfs_backend
$ MEMFS_BACKEND=~/.memfs_backend ./cryptfs_xor ~/memfs_mount -f -d
# another terminal
$ echo "secret" > ~/memfs_mount/a.txt
$ cat ~/memfs_mount/a.txt
secret
$ cat ~/.memfs_backend/a.txt
)?9(?P
$ printf "!" >> ~/memfs_mount/a.txt
$ cat ~/memfs_mount/a.txt
secret!
$ rm ~/memfs_mount/a.txt
$ ls ~/.memfs_backend
$ fusermount -u ~/memfs_mount

```

图 3 附加挑战二运行记录：挂载点明文、后端密文、删除同步

六、实验总结

本实验说明，普通命令并不需要知道底层文件系统类型。ls、cat、echo、rm 等命令仍然发起常规系统调用，VFS 根据挂载点把请求交给 FUSE，再由用户态回调函数完成路径判断、目录列出、数据读写和删除等操作。

基础 memfs 用内存数组实现了一个最小可读写文件系统，重点在于理解回调之间的分工和 offset 的语义；进阶多级目录版本把文件表扩展为节点树，补充了父子关系和目录错误处理；透明加密透传版本进一步说明 FUSE 不仅能构造内存文件系统，也能包装真实目录并在读写路径上增加自定义逻辑。

本实验仍有明显局限：文件内容只存在内存或简化后端中，没有真正日志、崩溃恢复、权限继承、并发锁和动态空间分配。若要接近真实文件系统，需要继续完善 inode 编号、引用计数、权限检查、目录项缓存、锁机制和持久化布局。

附录：提交包文件说明

文件	说明
memfs.c	基础任务：根目录普通文件可读写内存文件系统
memfs_tree.c	进阶挑战一：多级目录内存文件系统
cryptfs_xor.c	附加挑战二：XOR 透明加密透传文件系统
meson.build	Meson 构建文件
test_commands.sh	基础任务与两个进阶任务的验收命令
screenshots/*.png	报告中使用的运行记录截图